

Test problem 1: Channel flow (Poiseuille flow)

Contents

- Implementation
- Verification
- Visualization of vectors
- References

Authors: Anders Logg and Hans Petter
Langtangen

Adapted to DOLFINx by: Jørgen S. Dokken

In this section, you will learn how to:

- Solve the Navier-Stokes problem using a splitting scheme
- Visualize functions from higher order Lagrangian spaces

In this section, we will compute the flow between two infinite plates, so-called channel or Poiseuille flow. As we shall see, this problem has an analytical solution. Let H be the distance between the plates and L the

length of the channel. There are no body forces.

We may scale the problem first to get rid of seemingly independent physical parameters. The physics of this problem are governed by viscous effects only, in the direction perpendicular to the flow, so a time scale should be based on diffusion across the channel: $t_v = H^2/\nu$. We let U , some characteristic inflow velocity, be the velocity scale and H the spatial scale. The pressure scale is taken as the characteristic shear stress, $\mu U/H$, as this is a primary example of shear flow. Inserting $\bar{x} = x/H, \bar{y} = y/H, \bar{z} = z/H, \bar{u} = u/U, \bar{p}$, and $\bar{t} = H^2/\nu$ in the equations results in the scaled Navier-Stokes equations (dropping the bars after scaling)

$$\begin{aligned} \frac{\partial u}{\partial t} + \text{Re} u \cdot \nabla u &= -\nabla p + \nabla^2 u, \\ \nabla \cdot u &= 0. \end{aligned} \quad (37)$$

A detailed derivation for scaling of the Navier-Stokes equation for a large variety of physical situations can be found in [\[LP16\]](#) (Chapter 4.2) by Hans Petter Langtangen and Geir K. Pedersen.

Here, $\text{Re} = \rho U H / \mu$ is the Reynolds number. Because of the time and pressure scales, which are different from convection dominated fluid flow, the Reynolds number is associated with the convective term and not the viscosity term.

The exact solution is derived by assuming $u = (u_x(x, y, z), 0, 0)$ with the x -axis pointing along the channel. Since $\nabla \cdot u = 0$, u cannot be dependent on x .

The physics of channel flow is also two-dimensional so we can omit the z -coordinate (more precisely: $\partial/\partial z = 0$). Inserting $u = (u_x, 0, 0)$ in the (scaled) governing equations gives $u_x''(y) = \frac{\partial p}{\partial x}$. Differentiating this equation with respect to x shows that $\frac{\partial^2 p}{\partial x^2} = 0$ so $\partial p/\partial x$ is a constant here called $-\beta$. This is the driving force of the flow and can be specified as a known parameter in the problem. Integrating $u_x''(x, y) = -\beta$ over the width of the channel, $[0, 1]$, and requiring $u = (0, 0, 0)$ at the channel walls, results in $u_x = \frac{1}{2}\beta y(1 - y)$. The characteristic inlet velocity U can be taken as the maximum inflow at $y = 0.5$, implying $\beta = 8$. The length of the channel, L/H in the scaled model, has no impact on the result, so for simplicity we just compute on the unit square.

Mathematically, the pressure must be prescribed at a point, but since p does not depend on y , we can set p to a known value, e.g. zero, along the outlet boundary $x = 1$. The result is $p(x) = 8(1 - x)$ and $u_x = 4y(1 - y)$.

The boundary conditions can be set as $p = 8$ at $x = 0$, $p = 0$ at $x = 1$ and $u = (0, 0, 0)$ on the walls $y = 0, 1$. This defines the pressure drop and should result in unit maximum velocity at the inlet and outlet and

a parabolic velocity profile without no further specifications. Note that it is only meaningful to solve the Navier-Stokes equations in 2D or 3D geometries, although the underlying mathematical problem collapses to two $1D$ problems, one for $u_x(y)$ and one for $p(x)$.

The scaled model is not so easy to simulate using a standard Navier-Stokes solver with dimensions. However, one can argue that the convection term is zero, so the Re coefficient in front of this term in the scaled PDEs is not important and can be set to unity. In that case, setting $\rho = \mu = 1$ in the original Navier-Stokes equations resembles the scaled model.

For a specific engineering problem one wants to simulate a specific fluid and set corresponding parameters. A general solver is therefore most naturally implemented with dimensions and using the original physical parameters. However, scaling may greatly simplify numerical simulations. First of all, it shows that all fluids behave in the same way; it does not matter whether we have oil, gas, or water flowing between two plates. Secondly, it does not matter how fast the flow is, up to some critical value of the Reynolds number where the flow becomes unstable and transitions to a complicated turbulent flow of totally different nature. This means that one simulation is enough to cover all types of channel flow! In other applications, scaling shows that it might be necessary to

just set the fraction of some parameters (dimensionless numbers) rather than the parameters themselves. This simplifies exploring the input parameter space which is often the purpose of simulation. Frequently, the scaled problem is run by setting some of the input parameters with dimension to fixed values (often unity).

Implementation

Author: Jørgen S. Dokken

As in the previous example, we load the DOLFINx module, along with the `mpi4py` module, and create the unit square mesh and define the run-time and temporal discretization

```

from mpi4py import MPI
from petsc4py import PETSc
import numpy as np
import pyvista

from dolfinx.fem import (
    Constant,
    Function,
    extract_function_spaces,
    functionspace,
    assemble_scalar,
    dirichletbc,
    form,
    locate_dofs_geometrical,
)
from dolfinx.fem.petsc import (
    assemble_matrix,
    assemble_vector,
    apply_lifting,
    create_vector,
    set_bc,
)
from dolfinx.io import VTXWriter
from dolfinx.mesh import create_unit_
from dolfinx.plot import vtk_mesh
from basix.ufl import element
from ufl import (
    FacetNormal,
    Identity,
    TestFunction,
    TrialFunction,
    div,
    dot,
    ds,
    dx,
    inner,
    lhs,
    nabla_grad,
    rhs,
    sym,
)

mesh = create_unit_square(MPI.COMM_WO
t = 0.0
T = 10.0
num_steps = 500
dt = T / num_steps

```

As opposed to the previous demos, we will create our two function spaces using the

`ufl` element definitions as input

```
v_cg2 = element("Lagrange", mesh.basis)
s_cg1 = element("Lagrange", mesh.basis)
V = functionspace(mesh, v_cg2)
Q = functionspace(mesh, s_cg1)
```

The first space `V` is a vector valued function space for the velocity, while `Q` is a scalar valued function space for pressure. We use piecewise quadratic elements for the velocity and piecewise linear elements for the pressure. One can easily create vector-valued function spaces with other dimensions by replacing `shape=(mesh.geometry.dim, )` with something else, like

```
v_cg = basis.ufl.element("Lagrange", m
```

or

```
tensor_element = basis.ufl.element("L
```

or

```
tensor_element = basis.ufl.element("L
```

Stable finite element spaces for the Navier-Stokes equation

It is well-known that certain finite element spaces are not *stable* for the Navier-Stokes equations, or even for the simpler Stokes equation. The prime example of an unstable pair of finite element spaces is to use first order degree continuous piecewise polynomials for both the velocity and the pressure. Using an unstable pair of spaces typically results in a solution with *spurious* (unwanted, non-physical) oscillations in the pressure solution. The simple remedy is to use continuous piecewise quadratic elements for the velocity and continuous piecewise linear elements for the pressure. Together, these elements form the so-called *Taylor-Hood* element. Spurious oscillations may occur also for splitting methods if an unstable element pair is used.

Since we have two different function spaces, we need to create two sets of trial and test functions:

```
u = TrialFunction(V)
v = TestFunction(V)
p = TrialFunction(Q)
q = TestFunction(Q)
```


As we have seen in [Linear elasticity problem](#) we can use Python-functions to create the different Dirichlet conditions. For this problem, we have three Dirichlet condition: First, we will set $u = 0$ at the walls of the channel, that is at $y = 0$ and $y = 1$. In this case, we will use

```
dolfinx.fem.locate_dofs_geometrical
```

```
def walls(x):
    return np.logical_or(np.isclose(x[1], 0), np.isclose(x[1], 1))

wall_dofs = locate_dofs_geometrical(V, walls)
u_noslip = np.array((0,)) * mesh.geometric_dofs(V, wall_dofs)
bc_noslip = dirichletbc(u_noslip, wall_dofs, V)
```

Second, we will set $p = 8$ at the inflow ($x = 0$)

```
def inflow(x):
    return np.isclose(x[0], 0)

inflow_dofs = locate_dofs_geometrical(V, inflow)
bc_inflow = dirichletbc(PETSc.ScalarType(8), inflow_dofs, V)
```

And finally, $p = 0$ at the outflow ($x = 1$). This will result in a pressure gradient that will accelerate the flow from the initial state with zero velocity. At the end, we collect the boundary conditions for the velocity and pressure in Python lists so we can easily access them in the following computation.

```
def outflow(x):
    return np.isclose(x[0], 1)

outflow_dofs = locate_dofs_geometrical
bc_outflow = dirichletbc(PETSc.Scalar
bcu = [bc_noslip]
bcp = [bc_inflow, bc_outflow]
```

We now move on to the definition of the three variational forms, one for each step in the IPCS scheme. Let us look at the definition of the first variational problem and the relevant parameters.

```
u_n = Function(V)
u_n.name = "u_n"
U = 0.5 * (u_n + u)
n = FacetNormal(mesh)
f = Constant(mesh, PETSc.ScalarType((
k = Constant(mesh, PETSc.ScalarType(d
mu = Constant(mesh, PETSc.ScalarType(
rho = Constant(mesh, PETSc.ScalarType
```

Usage of "dolfinx.fem.Constant"

Note that we have wrapped several parameters as constants. This is to reduce the compilation-time of the variational formulations. By wrapping them as a constant, we can change the variable

The next step is to set up the variational form of the first step. As the variational problem contains a mix of known and unknown quantities, we will use the following naming convention: $\boxed{u}$ (mathematically u^{n+1}) is

known as a trial function in the variational form. `u_` is the most recently computed approximation (u^{n+1} available as a `Function` object), `u_n` is u^n , and the same convention goes for `p, p_` (p^{n+1}) and `p_n` (p^n).

```
def epsilon(u):
    """Strain-rate tensor."""
    return sym(nabla_grad(u))

def sigma(u, p):
    """Stress tensor."""
    return 2 * mu * epsilon(u) - p *
```

Define the variational problem for the first step

```
p_n = Function(Q)
p_n.name = "p_n"
F1 = rho * dot((u - u_n) / k, v) * dx
F1 += rho * dot(dot(u_n, nabla_grad(u
F1 += inner(sigma(u, p_n), epsilon(v)
F1 += dot(p_n * n, v) * ds - dot(mu *
F1 -= dot(f, v) * dx
a1 = form(lhs(F1))
L1 = form(rhs(F1))
```

Note that we have used the `ufl`-functions `lhs` and `rhs` to sort out the bilinear form $a(u, v)$ and linear form $L(v)$. This is particularly convenient in longer and more complicated variational forms. With our particular discretization $a(u, v)$ `a1` is not time dependent, and only has to be assembled once, while the right hand side is dependent on the solution from the previous time step (`u_n`). Thus, we do as for the [A](#)

[known analytical solution](#), and create the matrix outside the time-loop.

```
A1 = assemble_matrix(a1, bcs=bcu)
A1.assemble()
b1 = create_vector(extract_function_s
```

We now set up similar variational formulations and structures for the second and third step

```
# Define variational problem for step
u_ = Function(V)
a2 = form(dot(nabla_grad(p), nabla_grad(p_n)))
L2 = form(dot(nabla_grad(p_n), nabla_grad(p_n)))
A2 = assemble_matrix(a2, bcs=bcp)
A2.assemble()
b2 = create_vector(extract_function_spaces(u_))

# Define variational problem for step
p_ = Function(Q)
a3 = form(rho * dot(u, v) * dx)
L3 = form(rho * dot(u_, v) * dx - k * dot(u, u))
A3 = assemble_matrix(a3)
A3.assemble()
b3 = create_vector(extract_function_spaces(p_))
```

As we have created all the linear structures for the problem, we can now create a solver for each of them using PETSc. We can therefore customize the solution strategy for each step. For the tentative velocity step and pressure correction step, we will use the Stabilized version of BiConjugate Gradient to solve the linear system, and using algebraic multigrid for preconditioning. For the last step, the velocity update, we use a conjugate gradient method with successive over relaxation, Gauss Seidel (SOR) preconditioning.

```
# Solver for step 1
solver1 = PETSc.KSP().create(mesh.comm)
solver1.setOperators(A1)
solver1.setType(PETSc.KSP.Type.BCGS)
pc1 = solver1.getPC()
pc1.setType(PETSc.PC.Type.HYPRE)
pc1.setHYPREType("boomeramg")

# Solver for step 2
solver2 = PETSc.KSP().create(mesh.comm)
solver2.setOperators(A2)
solver2.setType(PETSc.KSP.Type.BCGS)
pc2 = solver2.getPC()
pc2.setType(PETSc.PC.Type.HYPRE)
pc2.setHYPREType("boomeramg")

# Solver for step 3
solver3 = PETSc.KSP().create(mesh.comm)
solver3.setOperators(A3)
solver3.setType(PETSc.KSP.Type.CG)
pc3 = solver3.getPC()
pc3.setType(PETSc.PC.Type.SOR)
```

We prepare output files for the velocity and pressure data, and write the mesh and initial conditions to file

```
from pathlib import Path

folder = Path("results")
folder.mkdir(exist_ok=True, parents=True)
vtx_u = VTXWriter(mesh.comm, folder / "u.vtx")
vtx_p = VTXWriter(mesh.comm, folder / "p.vtx")
vtx_u.write(t)
vtx_p.write(t)
```

We also interpolate the analytical solution into our function-space and create a variational formulation for the L^2 -error.

```
def u_exact(x):
    values = np.zeros((2, x.shape[1]))
    values[0] = 4 * x[1] * (1.0 - x[1])
    return values

u_ex = Function(V)
u_ex.interpolate(u_exact)

L2_error = form(dot(u_ - u_ex, u_ - u_ex))
```

The next step is to create the loop over time. Note that we for all three steps only have to assemble the right hand side and apply the boundary condition using lifting.

```
for i in range(num_steps):
    # Update current time step
    t += dt

    # Step 1: Tentative velocity step
    with b1.localForm() as loc_1:
        loc_1.set(0)
    assemble_vector(b1, L1)
    apply_lifting(b1, [a1], [bcu])
    b1.ghostUpdate(addv=PETSc.InsertMode, mode=1)
    set_bc(b1, bcu)
    solver1.solve(b1, u_.x.petsc_vec)
    u_.x.scatter_forward()

    # Step 2: Pressure correction step
    with b2.localForm() as loc_2:
        loc_2.set(0)
    assemble_vector(b2, L2)
    apply_lifting(b2, [a2], [bcp])
    b2.ghostUpdate(addv=PETSc.InsertMode, mode=1)
    set_bc(b2, bcp)
    solver2.solve(b2, p_.x.petsc_vec)
    p_.x.scatter_forward()

    # Step 3: Velocity correction step
    with b3.localForm() as loc_3:
        loc_3.set(0)
    assemble_vector(b3, L3)
    b3.ghostUpdate(addv=PETSc.InsertMode, mode=1)
    solver3.solve(b3, u_.x.petsc_vec)
    u_.x.scatter_forward()
    # Update variable with solution f
```

```

u_n.x.array[:] = u_.x.array[:]
p_n.x.array[:] = p_.x.array[:]

# Write solutions to file
vtx_u.write(t)
vtx_p.write(t)

# Compute error at current time-s
error_L2 = np.sqrt(mesh.comm.allreduce(
    np.max(u_.x.petsc_vec.array -
)
)
# Print error only every 20th step
if (i % 20 == 0) or (i == num_steps):
    print(f"Time {t:.2f}, L2-error {error_L2:.2e}")

# Close xmdf file
vtx_u.close()
vtx_p.close()
b1.destroy()
b2.destroy()
b3.destroy()
solver1.destroy()
solver2.destroy()
solver3.destroy()

```

```

Time 0.02, L2-error 5.88e-01, Max error 5.88e-01
Time 0.42, L2-error 1.09e-02, Max error 1.09e-02
Time 0.82, L2-error 2.11e-04, Max error 2.11e-04
Time 1.22, L2-error 1.98e-05, Max error 1.98e-05
Time 1.62, L2-error 9.53e-06, Max error 9.53e-06
Time 2.02, L2-error 5.89e-06, Max error 5.89e-06
Time 2.42, L2-error 4.55e-06, Max error 4.55e-06
Time 2.82, L2-error 3.99e-06, Max error 3.99e-06
Time 3.22, L2-error 3.71e-06, Max error 3.71e-06
Time 3.62, L2-error 3.55e-06, Max error 3.55e-06
Time 4.02, L2-error 3.46e-06, Max error 3.46e-06
Time 4.42, L2-error 3.41e-06, Max error 3.41e-06
Time 4.82, L2-error 3.37e-06, Max error 3.37e-06

```

```

Time 5.22, L2-error 3.35e-06, Max error 3.35e-06
Time 5.62, L2-error 3.34e-06, Max error 3.34e-06
Time 6.02, L2-error 3.33e-06, Max error 3.33e-06
Time 6.42, L2-error 3.32e-06, Max error 3.32e-06
Time 6.82, L2-error 3.32e-06, Max error 3.32e-06
Time 7.22, L2-error 3.32e-06, Max error 3.32e-06
Time 7.62, L2-error 3.31e-06, Max error 3.31e-06

```

```

Time 8.02, L2-error 3.31e-06, Max error
Time 8.42, L2-error 3.31e-06, Max error
Time 8.82, L2-error 3.31e-06, Max error
Time 9.22, L2-error 3.31e-06, Max error
Time 9.62, L2-error 3.31e-06, Max error
Time 10.00, L2-error 3.31e-06, Max error

```

```
<petsc4py.PETSc.KSP at 0x7f4c24779b20:
```

Verification

As for the previous problems we compute the error at each degree of freedom and the $L^2(\Omega)$ -error. We start with the initial condition $u = (0, 0)$. We have not specified the initial condition explicitly, and FEniCSx will initialize all `Function`s including `u_n` and `u_` to zero. Since the exact solution is quadratic, we expect to reach machine precision within finite time. For our implementation, we observe that the error quickly approaches zero, and is of order 10^{-6} at $T = 10$

Visualization of vectors

We have already looked at how to plot higher order functions and vector functions. In this section we will look at how to visualize vector functions with glyphs, instead of warping the mesh.


```

topology, cell_types, geometry = vtk_
values = np.zeros((geometry.shape[0],
values[:, : len(u_n)] = u_n.x.array.r

# Create a point cloud of glyphs
function_grid = pyvista.UnstructuredG
function_grid["u"] = values
glyphs = function_grid.glyph(orient="

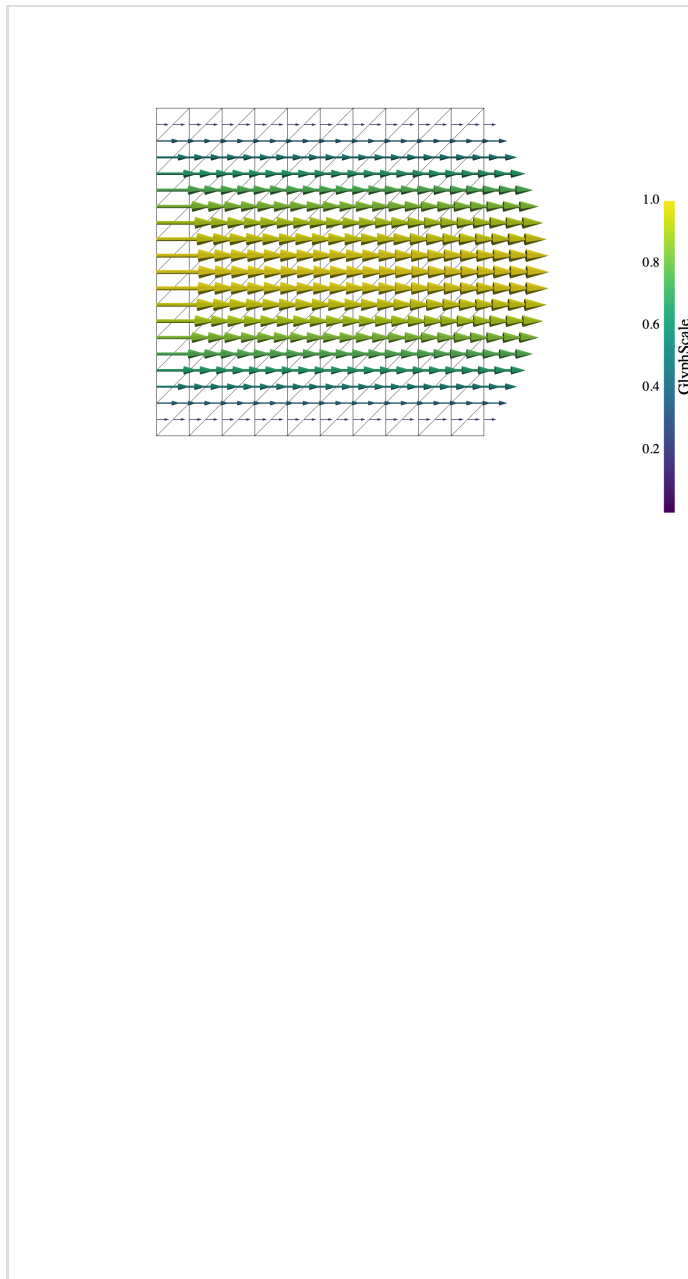
# Create a pyvista-grid for the mesh
tdim = mesh.topology.dim
mesh.topology.create_connectivity(tdi
grid = pyvista.UnstructuredGrid(*vtk_

# Create plotter
plotter = pyvista.Plotter()
plotter.add_mesh(grid, style="wirefra
plotter.add_mesh(glyphs)
plotter.view_xy()

if not pyvista.OFF_SCREEN:
    plotter.show()
else:
    fig_as_array = plotter.screenshot

```

2026-05-18 09:06:25.799 (4.030s) [



References

- [[LP16](#)] Hans Petter Langtangen and Geir K. Pedersen. *Advanced partial differential equation models*, pages 99–134. Springer International Publishing, Cham, 2016.
[doi:10.1007/978-3-319-32726-6_4](https://doi.org/10.1007/978-3-319-32726-6_4).